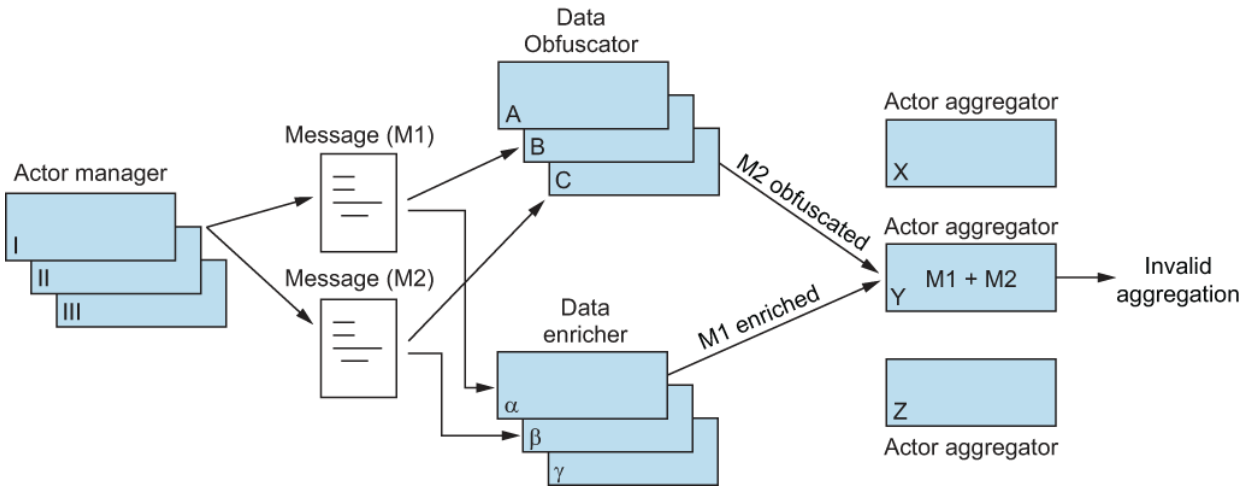


Figure 6.6 Distributing and aggregating messages

To solve this problem—among others—the *consistent hashing* strategy was introduced. Consistent hashing is an algorithm that solves a more general problem than this and is widely used in the industry to partition data. Examples include Cassandra, DynamoDB, Couchbase, GlusterFS, and many more. The history of the algorithm goes back to trying to balance the load on cache servers on the internet, taking into account that servers go down and new ones come in, and some servers can handle a larger load than others. This is one of the reasons why in Akka, *consistent hashing* is usually used with the group router and not with the pool router. The routees in a group router are meant to increase or decrease, register or deregister, but the same is not true in a pool router.

Returning to the previous example, to send messages generated from the same original message to the same aggregator, you can use a router with consistent hashing in the data-obfuscator and data-enricher actors. An instance of this router in each worker sends similar messages to the same routee and to the same aggregator, allowing the pieces to fit together properly.